Software Verification Plan (SVP)

Project Title: *MataBank* Banking Website

Date: 6 April 2025

(Note: Highlighted Text = Revisions Made)

Lernik Avetisyan, Gus Axelson, Lance Jimenez, Anthony Taylor

1. **Introduction**

The purpose of a Software Verification Plan (or SVP) is to ensure that our software, *MataBank* Banking Website, successfully fulfills customer specifications by verifying the proper performance of the major functions. The software will offer a web-based platform for users to manage their finances, providing features such as secure logins, and money transfers.

---

2. **Objectives**

The objectives of this Software Verification Document are to confirm that the functional and non-functional requirements are correctly implemented, to determine if there are any defects within the system and correct them, and to ensure the reliability and performance of the system.

---

3. **Scope**

This Software Verification Plan covers both functional and non-functional aspects of the *MataBank* Banking Website.

Functional testing: focus on verifying key features such as user login, account balance viewing, money transfers, and transaction history. These features will be tested individually, together, and as a full system to ensure everything works correctly and reliably.

Non-functional testing: includes checks for security, performance, and usability. This involves testing secure authentication, encrypted data transfer, system responsiveness under load, and overall user experience.

The goal is to confirm that the website not only functions as expected, but also performs securely and efficiently for users.

---

4. **Verification Approach**

Our approach consists of a series of tests at varying levels on our project which test the reliability, security and performance of our banking application.

Testing Levels:

A. **Unit Testing**
   We will begin by individually testing components such as the login system, account balance calculator, and the transfer handler to verify that it behaves as expected.

Since we're working with JavaScript, we plan to use frameworks like Jest or Mocha rather than JUnit because that is specifically for java programs.

B. **Integration Testing**
   Once the individual components are verified, we will test how they work together in integration testing. For example, we will test how the frontend login form communicates with the backend authentication system. This verifies that the individual units work smoothly in combination.
C. **System Testing**
   After integration testing, we will conduct full system tests to validate the end to end functionality of the website. This includes testing features like user registration, login, fund transfers, and viewing transaction history. Making sure all features work together as a whole.

D. **User Acceptance Testing**
   Finally, we will involve real users to interact with the system and provide feedback. This will help confirm that the application meets user expectations and business requirements.

Testing Methods:

A. **Black Box Testing**
   We will use black box testing, testing as users without knowledge of the system, to validate that the system produces the correct outputs for various inputs. For instance, we will verify that entering valid credentials logs the user in and invalid ones trigger appropriate error messages.
B. **White Box Testing**
   We as the developers will perform white box testing, having full understanding of the system, during unit testing by examining internal logic, loops, and conditions within the code, ensuring our logic is correctly implemented.
C. **Security Testing**
   Ensure secure password handling, encrypted data transmission over HTTPS, third-party multi factor authentication, and proper session management using technologies like OWASP ZAP.
D. **Performance Testing**
   We will simulate multiple users accessing the site at once to measure how well it performs under load by testing the speed of login, dashboard loading, and transfers. Any bottlenecks will be addressed to ensure a responsive product.

---

5. **Verification Environment**

The environment to verify our project is a very important stage in maintaining consistent, reproducible results. Listed below are the specifications for the verification hardware, software and the tools used.

Hardware Specs:

- Processor: Intel Core i5 (or equivalent)
- RAM: 8GB
- Storage: 256GB
- Operating System: Windows 10 / macOS / Linux

Software

- **Web Browser**: Google Chrome (latest version) – for frontend testing and developer tool
- **Code Editor**: Visual Studio Code – for reviewing and testing frontend/backend code
- **Node.js & NPM**:
    - Node.js (v18 or above): runtime environment
    - NPM: for managing JavaScript dependencies and testing libraries
- **Local Web Server**: Express.js: to run backend APIs locally for integration and system testing
- **Database**: MySQL: for storing and verifying user and transaction data

Testing Tools

- **Unit & Integration Testing**:
    - **Jest or Mocha**: used for unit testing JavaScript functions and modules
    - **Supertest**: for testing API endpoints during integration testing
- **System & End-to-End Testing**:
    - **Cypress**: to automate full workflows like login, fund transfer, and viewing transaction history
- **Security Testing**:
    - **OWASP ZAP (light mode)**: to perform lightweight scans for common web vulnerabilities
- **Performance Testing**:
    - **Lighthouse (Chrome DevTools)**: to evaluate site speed, accessibility, and performance

---

6. **Test Cases**

Test Case ID	Test Description	Expected Outcome	Test Type	Status
TC1	User registration with valid credentials	Successful Registration	Functional	Pending
TC2	Login with unregistered	Display Error	Functional	Pending

	credentials	Message		
TC3	Deposit and Withdrawals of valid amount (login->deposit/withdraw ->input the amount to be moved->confirm)	Deposit/Withdrawal Confirmed, Balance Updated	Functional	Pending
TC4	Withdrawals of invalid amount (login->withdraw->input the amount to be moved->confirm)	Insufficient Funds	Functional	Pending
TC5	Money Transfer between two valid parties (login->money transfer->enter information of receiving party->confirm amount->receiving party accepts)	Confirmed Transfer, Balance Updated	Functional	Pending
TC6	Money Transfer to an invalid party (login->money transfer->enter information of receiving party->confirm amount)	Display Error Message	Functional	Pending
TC7	View Balance (login->view balance)	Correct Balance Displayed	Functional	Pending
TC8	Login Security Measures (attempt to login->TFA entry required)	One Time Password or Security Question Required	Security	Pending
TC9	Encryption of Sensitive Data	Data Encrypted	Security	Pending
TC10	Role-Based Access Control	Appropriate access for individual Roles	Security	Pending

---

7. **Defect Management**

Defect management is how we track and fix problems or bugs in the software. It helps us know what is wrong and how to correct it before the product is released.

How we do it:

- **Logging Defects:** When a tester finds a problem, they record it in a tool (like JIRA or Bugzilla) with a simple description, steps to repeat the problem, and a screenshot if needed.
- **Severity Levels:** We sort problems by importance:
  - **Critical:** Must fix immediately because it can crash the system or cause data loss.
  - **Major:** Affects main functions but might have a workaround.
  - **Minor:** Small issues that do not stop the system from working.
  - **Trivial:** Very small problems or cosmetic issues.
- P**rocess:**
  - **Report:** Tester writes down the issue.
  - **Assign:** The problem is given to a developer.
  - **Fix:** The developer corrects the problem.
  - **Verify:** Tester checks if the fix works.
  - **Close:** Once fixed and verified, the defect is marked as closed.

---

8. **Roles and Responsibilities**

There will be several key roles involved during the planning and execution:

- **Test Lead:** **Gus Axelson**

  Responsible for planning test cases with consideration to customer requirements and specifications
- **Test Engineers:** **Anthony Taylor**

  Responsible for manufacturing and executing test cases, including deciding whether the test was a successful or not
- **Developers:** **Lernik Avetisyan**

Responsible for adjusting the software in accordance with the results of each test case

- **Business Analysts:** Lance Jimenez

  Responsible for verifying that each test case proposed is relevant to the customer's needs

---

9. Schedule and Milestones

○	**Test Planning:** Define testing scope and objectives	Week 9
○	**Test Case Design:** Identify and write test cases	Week 10
○	**Unit Testing:** Execute unit testing and code level testing	Week 11
○	**Integration Testing:** Verify API and module integration	Week 12
○	**System Testing:** Perform end to end system testing validate security, performance, and compliance	Week 13
○	**User Acceptance Testing:** Confirm website meets user needs	Week 14
○	**Final Verification and Reporting:** Prepare test report	Week 15

---

10. Verification Deliverables

Verification deliverables are the documents and reports that prove we have tested the software and that it works as expected.

Key Deliverables:

- **Test Plan:** A document that explains what we will test, how we will test it, and what the goals are.
- **Test Cases and Test Scripts:** Detailed instructions (in a table) showing what steps to perform, what the expected results are, and what type of test it is (like functional or negative testing).
- **Test Data:** The sample data we use during testing, including both valid and invalid examples.
- **Test Reports:** Summaries of what tests were done, the results (pass or fail), and any problems found.
- **Defect Logs:** A record of all the bugs we found, their severity, and how they were fixed.
- **Traceability Matrix:** A table that links each requirement to its corresponding test cases, ensuring every requirement is tested
- **Verification Summary:** A final report that summarizes the whole testing process and the state of the software before release.

---

11. Risk and Mitigation

Risk and mitigation are about identifying what might go wrong during testing and planning ways to reduce or handle those problems

Common risks and how we can deal with them:

- **Incomplete Test Coverage:**

  Risk: Some features or requirements might not be tested.

  Mitigation: We use a traceability matrix to make sure every requirement has matching test cases.

- **Too Many Defects:**

  Risk: If many bugs are found, they can delay the project.

  Mitigation: We prioritize fixes by severity and have regular meetings to review and address critical defects first.

- **Unstable Test Environment:**

  Risk: Problems with our test setup might hide true software issues.

  Mitigation: We maintain a dedicated and stable test environment that mirrors the real (production) environment as closely as possible.

- **Resource or Time Constraints:**

  Risk: Limited time or team members might cause rushed testing.

  Mitigation: We plan realistic schedules, automate repetitive tests, and set clear roles and responsibilities.

- **Security Vulnerabilities:**

  Risk: Undetected security issues could lead to data breaches.

  Mitigation: We include basic security testing and, if needed, ask for expert reviews to help find security flaws.

---

12. Conclusion

The Software Verification Plan allows the software's functionality to tested and verified while clarifying the overall purpose for the creation of the *MataBank* Banking Website. Closely following the SVP and successfully executing it will guarantee a properly assessed product that will serve the customer's needs.